



Higher Diploma in Computer Science

Computer Systems & Networks





**Positional
Parameters;
Exit codes**

```
#!/bin/bash  
variable=value
```



Variables: RECAP

Variables

- Variables are a key part of any programming language.
- A very simple use of variables is shown here:

```
echo Enter your first number:  
read first  
echo Enter your second number:  
read second  
z=`expr $first + $second`  
echo The sum is ${z}
```

Variables

- Another way to assign to a variable is to use the output of another command as the contents of the variable by enclosing the command in backtick ` characters.:

```
#!/bin/bash
CURRENT_DIRECTORY=`pwd`
echo "You are in $CURRENT_DIRECTORY"
```

- It is also possible to get input from the user of the script and assign that input to a variable using the `read` command:

```
echo -n "What is your name?"
read name
echo "Hello ${name}! Nice of you to join us"
```

What syntax is missing that causes an error when executed?

Variables: Summary

- Don't need to declare

```
caroline@caroline-VirtualBox:~$ x=10
caroline@caroline-VirtualBox:~$ echo $x
```

- read command

```
caroline@caroline-VirtualBox:~$ read number
45
caroline@caroline-VirtualBox:~$ echo $number
```

- No type

```
caroline@caroline-VirtualBox:~$ x=8
caroline@caroline-VirtualBox:~$ y=5
caroline@caroline-VirtualBox:~$ z=$x+$y
caroline@caroline-VirtualBox:~$ echo $z
```

The output is:

- expr

```
x=8
y=5
z=$x+$y
echo $z
```

```
expr $x+$y
```

```
z=`expr $x + $y`
echo The sum is $z
```

Now try out
multiplying 6 by 4



Command Chaining: RECAP

Command Chaining

1st up, the ; (semicolon) → run one command right after another

```
$ls ; pwd ; whoami
```

2nd up, the && (and) → second command to only run if the first command is successful

```
$mkdir compsys2022 && cd compsys2022
```

With ; the second command will run even if the first one fails

Command Chaining

Check if a directory exists

`[ -d "/path/dir/" ]`

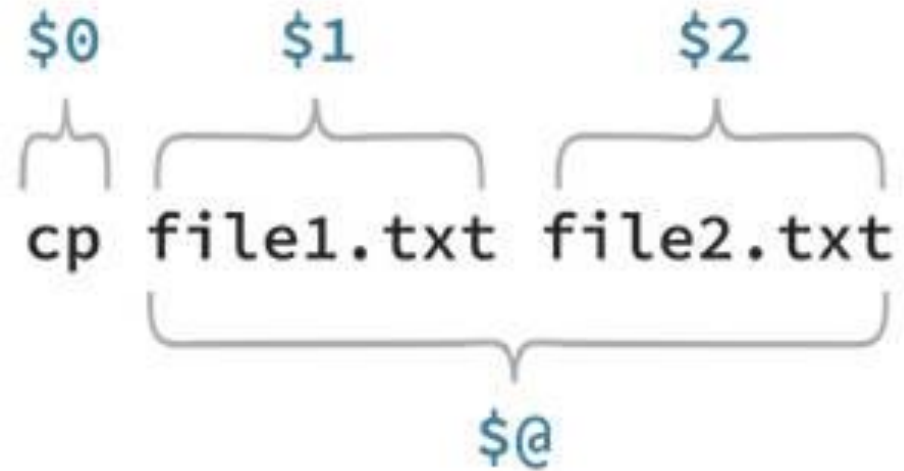
```
compsys@compsys-virtualbox:~$ [ -d "temp" ] && echo "Directory exists"
Directory exists
compsys@compsys-virtualbox:~$ [ ! -d "rainy" ] && echo "Directory DOESN'T exists"
Directory DOESN'T exists
```

3rd and last, the `||` (or)

→ Sometimes you want to execute second command only if first command *is not* successful

```
$ [ -d "newHDipFolder" ] && echo "Directory exists" || mkdir ~/newHDipFolder
```

- b filename - Block special file
- c filename - Special character file
- d directoryname - Check for directory Existence
- e filename - Check for file existence, regardless of type (node, directory, socket, etc.)
- f filename - Check for regular file existence not a directory
- G filename - Check if file exists and is owned by effective group ID
- G filename set-group-id - True if file exists and is set-group-id
- k filename - Sticky bit
- L filename - Symbolic link
- O filename - True if file exists and is owned by the effective user id
- r filename - Check if file is a readable
- S filename - Check if file is socket
- s filename - Check if file is nonzero size
- u filename - Check if file set-user-id bit is set
- w filename - Check if file is writable
- x filename - Check if file is executable



A diagram illustrating the mapping of command-line arguments to shell variables. The command `cp file1.txt file2.txt` is shown. Above the command, curly braces group the arguments: `cp` is grouped under `$0`, `file1.txt` is grouped under `$1`, and `file2.txt` is grouped under `$2`. A larger curly brace below `file1.txt` and `file2.txt` is labeled `$@`, representing all arguments starting from the first.

```
$0      $1      $2
┌──┐    ┌────────┐ ┌────────┐
cp  file1.txt file2.txt
    └────────┘
      $@
```

Positional Parameters

Variables → Positional Parameters

- The **\$0** variable contains the name of the script itself. Additionally, you can pass *arguments* to your script:

```
compsys@compsys-virtualbox:~$ cat pospa*  
#!/bin/bash  
echo -n "What's your name? "  
read name  
echo "Hello ${name}! You're looking at your file $0"
```

A dollar sign followed by a number *N* corresponds to the **Nth** argument passed to the script

Positional Parameters: \$

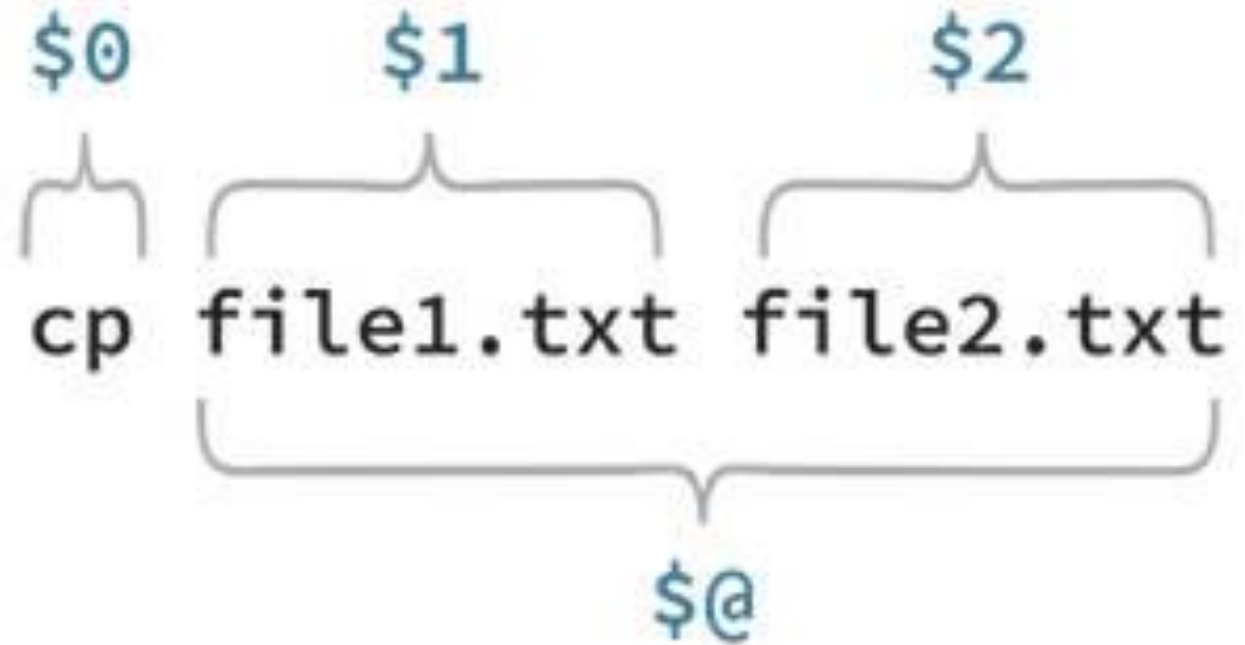
...referenced using the \$ sign.

\$1 variable is used to read the **first** command line argument

\$2 variable is used to read the **second** command line argument
etc.

(Two digits are written as **\${10}**,
\${11} and so on.)

\$0 is the **script name** or else the **command** given



Positional Parameters: **\$#**

\$# the number of positional parameters, excluding **\$0**.

\$# is often used to check that the user has entered the correct number of command line arguments were passed to the script.

Positional Parameters: **\$***

\$* references *all* the arguments passed to a program
(useful in programs that take an indefinite or variable number of arguments)



Exit Codes

Exit Codes

indicates the response from the command or a script after execution

- help us determine whether a process ran Successfully

For the bash shell's purposes, a command which exits with a:

- zero (0) exit status = **success**;
- non-zero (1-255) exit status = **failure**

command causes the shell or program to terminate

- You can set the exit code of your own script with the **exit** command.



When running commands at a shell prompt, the special variable `$?` contains a number that indicates the **result of the last command** executed

- A zero (0) means everything went fine. Anything else means there is a problem.
- A value of 1 generically indicates that some error has happened.

```
compsys@compsys-virtualBox:~$ cat employee.txt
compsys@compsys-virtualBox:~$ echo $?
0
compsys@compsys-virtualBox:~$ grep -q asdfjkl; /etc/passwd
compsys@compsys-virtualBox:~$ echo $?
1
```

This can be tested through the **\$?** variable to see if the previous command completed successfully (no errors=0)

```
if [ ! -f compsys.txt ];
```

```
then
```

```
    echo The file does not exist and I display an error message
```

```
    exit 23    #random code for error condition
```

```
fi
```

```
echo The file exists and we will do something else
```

```
echo "here you could have your script doing something"
```

```
exit 0
```

\$/myexit_test.sh

The file does not exist and I display an error message

\$echo \$?

64

\$touch compsys.txt

\$/ myexit_test.sh

**The file exists and we will do something else
here you could have your script doing something**

\$echo \$?

0

PRACTICE: Using positional parameters & redirection to append to fields in file

- Write a script that can be used to add in a new employees details to the **employee.txt** file

```
Caroline@ICTSkills>~$ cat employee.txt
100 frank manager sales €5000
200 caroline developer technology €5500
300 mary sysadmin technology €7000
400 rosanne manager marketing €9500
500 eamonn dba technology €6000
```

- What could be the **steps** you might take with this?